Linux TCP Socket API

A practical Slipstream study guide

Goal: understand exactly how socket(), bind(), listen(), accept(), connect(), poll(), send(), recv(), shutdown(), and close() cooperate - including non-blocking behavior and TCP stream framing.

```
market_data_client -- TCP port 9001 --> Slipstream
order_entry_client - TCP port 9002 --> Slipstream

Server: socket -> bind -> listen -> accept -> poll -> recv/send
Client: socket -> connect ---------------------> recv/send
```

Designed as a revisit-friendly reference for Milestone 2 of the Slipstream project.

1. The mental model

A socket is a kernel-managed communication endpoint. Your C++ process does not contain the TCP implementation; it holds an integer file descriptor that refers to a socket object inside the Linux kernel.

```
C++ Socket object
    `-- fd_ = 5
          `-- kernel TCP socket
                `-- peer connection
```

File descriptor numbers are local to a process. Slipstream fd 5 and a client fd 5 are unrelated numbers, even when those descriptors refer to opposite ends of the same TCP connection.

How many sockets are available?

There is no single fixed kernel socket count. Practical limits come from per-process file-descriptor limits, system-wide descriptor limits, available kernel memory, and available TCP address combinations.

```
ulimit -Sn                              # soft per-process FD limit
ulimit -Hn                              # hard per-process FD limit
cat /proc/sys/fs/file-max               # system-wide file limit
cat /proc/sys/net/ipv4/ip_local_port_range
```

A TCP connection is identified by source IP, source port, destination IP, and destination port. Thousands of clients can therefore share one server port because their source endpoints differ.

Slipstream descriptors

Descriptor role	Purpose
MD listener	Waits for connections on port 9001
OE listener	Waits for connections on port 9002
MD connection	Carries bytes to/from market_data_client
OE connection	Carries bytes to/from order_entry_client

> **Listening sockets accept connections. Connected sockets carry protocol bytes. accept() does not replace the listener; it returns a new connected descriptor.**

2. socket(): create an endpoint

```
int fd = ::socket(
    AF_INET,
    SOCK_STREAM | SOCK_CLOEXEC,
    0);
```

Argument	Meaning	Useful alternatives
AF_INET	IPv4	AF_INET6 for IPv6; AF_UNIX for local IPC
SOCK_STREAM	Reliable byte stream, normally TCP	SOCK_DGRAM for UDP; SOCK_SEQPACKET where supported
SOCK_CLOEXEC	Do not leak descriptor through exec()	SOCK_NONBLOCK may be ORed here
0	Default protocol for the family/type	IPPROTO_TCP can be explicit

A successful call returns the lowest available descriptor number. A failure returns -1 and sets errno. At this stage the TCP socket is neither bound, listening, nor connected.

Address family alternatives

Family	Address structure	Typical use
AF_INET	sockaddr_in	IPv4 networking - Slipstream's simplest choice
AF_INET6	sockaddr_in6	IPv6 networking
AF_UNIX	sockaddr_un	Same-machine IPC using a local path
AF_NETLINK	netlink structures	Userspace communication with Linux kernel facilities
AF_PACKET	sockaddr_ll	Raw link-layer packets; specialized/privileged

3. Server setup: setsockopt, bind, listen

SO_REUSEADDR before bind()

```
int enabled = 1;

if (::setsockopt(fd, SOL_SOCKET, SO_REUSEADDR,
                 &enabled, sizeof(enabled)) == -1) {
    throw ...;
}
```

SO_REUSEADDR relaxes address-reuse rules and helps a restarted server bind again. It does not mean that two ordinary active listeners can freely own the same endpoint.

sockaddr_in and byte order

```
sockaddr_in address{};
address.sin_family = AF_INET;
address.sin_port = htons(9001);
address.sin_addr.s_addr = htonl(INADDR_ANY);
```

- htons() means host-to-network 16-bit value. TCP/IP port fields use big-endian network byte order.
- INADDR_ANY means 0.0.0.0: bind on all local IPv4 interfaces.
- GCMD/GCOE payload fields remain little-endian. Address encoding and application protocol encoding are separate concerns.

bind() and listen()

```
if (::bind(fd,
           reinterpret_cast<const sockaddr*>(&address),
           sizeof(address)) == -1) {
    throw ...;
}

if (::listen(fd, 8) == -1) {
    throw ...;
}
```

bind() claims the local endpoint. listen() converts it into a TCP listening socket. The backlog concerns connections waiting to be accepted; it is not the maximum number of active clients.

accept() creates another socket

```
int connected_fd = ::accept(listener_fd, nullptr, nullptr);

// listener_fd still listens.
// connected_fd communicates with one client.
```

In an RAII design, Accept() returns a new move-only Socket that adopts connected_fd. The listener and returned connection own different descriptors.

4. Client setup: inet_pton and connect

```
sockaddr_in server{};
server.sin_family = AF_INET;
server.sin_port = htons(9001);

if (::inet_pton(AF_INET, "127.0.0.1", &server.sin_addr) != 1) {
    throw ...;
}

if (::connect(fd,
              reinterpret_cast<const sockaddr*>(&server),
              sizeof(server)) == -1) {
    throw ...;
}
```

inet_pton() only converts a numeric textual IP address into binary form. It does no networking. getaddrinfo() is the broader alternative for hostnames and IPv4/IPv6 candidates.

connect() error	Meaning
ECONNREFUSED	Nothing is listening at the destination
ETIMEDOUT	Connection attempt timed out
ENETUNREACH	No usable route to the network
EINTR	A signal interrupted the call

5. Exact Slipstream startup order

```
 1. Slipstream socket() for MD listener.
 2. Slipstream sets SO_REUSEADDR.
 3. Slipstream bind() to port 9001.
 4. Slipstream listen().

 5. Slipstream socket() for OE listener.
 6. Slipstream sets SO_REUSEADDR.
 7. Slipstream bind() to port 9002.
 8. Slipstream listen().

 9. MD client socket().
10. MD client connect() to 127.0.0.1:9001.
11. Slipstream accept() returns MD connection.

12. OE client socket().
13. OE client connect() to 127.0.0.1:9002.
14. Slipstream accept() returns OE connection.

15. Slipstream poll() watches both connected sockets.
16. Clients send encoded frames.
17. Slipstream recv() feeds per-connection stream decoders.
18. Slipstream sends GCOE responses on OE connection.
19. Either client disconnects.
20. Slipstream closes the other connection and reports results.
```

The two TCP streams are independent. Never feed bytes from both connections into one decoder, and do not assume equal timestamps across separate connections arrive in a deterministic order.

6. Non-blocking mode

```
int flags = ::fcntl(fd, F_GETFL, 0);
if (flags == -1) {
    throw ...;
}

if (::fcntl(fd, F_SETFL, flags | O_NONBLOCK) == -1) {
    throw ...;
}
```

Preserve existing flags when adding O_NONBLOCK. Alternatively, create the socket with SOCK_NONBLOCK. Non-blocking means an operation returns instead of sleeping when it cannot progress immediately.

EAGAIN or EWOULDBLOCK means 'nothing can happen right now'. It is normal control flow, not a disconnected peer and not a permanent error.

Non-blocking connect

```
connect(fd, ...)
    |-- returns 0 ------------------------> connected
    |-- errno != EINPROGRESS ------------> failed
    `-- errno == EINPROGRESS
            |
            v
       poll(fd, POLLOUT, timeout)
            |
            v
       getsockopt(fd, SOL_SOCKET, SO_ERROR, ...)
            |-- SO_ERROR == 0 -----------> connected
            `-- SO_ERROR != 0 -----------> failed
```

POLLOUT only says the connection attempt finished. It does not prove success. SO_ERROR contains the real result. If nonzero, use that value as the error code rather than the current errno.

```
int socket_error = 0;
socklen_t size = sizeof(socket_error);

if (::getsockopt(fd, SOL_SOCKET, SO_ERROR,
                 &socket_error, &size) == -1) {
    throw ...;
}

if (socket_error != 0) {
    throw std::system_error(socket_error,
                            std::generic_category(),
                            "connect");
}
```

Non-blocking accept

```
int connected_fd = ::accept4(
    listener_fd,
    nullptr,
    nullptr,
    SOCK_NONBLOCK | SOCK_CLOEXEC);
```

After poll() reports POLLIN on a listener, accept queued connections. EAGAIN/EWOULDBLOCK means the accept queue is currently empty. accept4() atomically applies useful flags to the returned connection.

7. poll(): monitor multiple descriptors

```
pollfd descriptors[] = {
    {.fd = md_fd, .events = POLLIN, .revents = 0},
    {.fd = oe_fd, .events = POLLIN, .revents = 0}
};

int ready = ::poll(descriptors, 2, -1);
```

Event	Meaning
POLLIN	Reading can make progress
POLLOUT	Writing can make progress, or non-blocking connect completed
POLLERR	A socket error exists
POLLHUP	Connection hangup
POLLRDHUP	Peer closed its sending side
POLLNVAL	Descriptor is invalid

Only request POLLOUT while output is pending. A connected TCP socket is commonly writable, so permanent POLLOUT interest can make the event loop wake continuously.

Multiple clients

Every accept() returns a new connected descriptor. A server stores one decoder and one pending-output queue per connection, then lets poll() or epoll report which descriptors are ready.

```
struct Connection {
    Socket socket;
    StreamDecoder decoder;
    std::vector<std::byte> pending_output;
};

std::unordered_map<int, Connection> connections;
```

poll() is sufficient for Slipstream's two client connections. epoll is the usual Linux choice when managing very large descriptor sets.

8. read/write versus recv/send

Calls	Scope	Key characteristic
read / write	Generic file descriptors	Files, pipes, terminals, devices, and sockets
recv / send	Sockets	Adds socket-specific per-call flags

```
read(fd, buffer, size)       ~= recv(fd, buffer, size, 0)
write(fd, bytes, size)       ~= send(fd, bytes, size, 0)
```

For connected TCP and zero flags, the operations are effectively equivalent. Slipstream should use send()/recv() because it benefits from MSG_NOSIGNAL and MSG_DONTWAIT.

send() and partial writes

```
ssize_t sent = ::send(
    fd,
    bytes.data() + offset,
    bytes.size() - offset,
    MSG_NOSIGNAL | MSG_DONTWAIT);
```

- MSG_NOSIGNAL prevents SIGPIPE from terminating the process. EPIPE is still returned.
- MSG_DONTWAIT makes only this call non-blocking; O_NONBLOCK affects the descriptor persistently.
- A positive result can be smaller than the requested size. Advance offset and retain the unsent suffix.
- On EAGAIN/EWOULDBLOCK, request POLLOUT and retry the remaining bytes later.

recv() and the three outcomes

```
ssize_t received = ::recv(
    fd,
    buffer.data(),
    buffer.size(),
    MSG_DONTWAIT);
```

Result	Meaning	Action
> 0	That many bytes arrived	Feed exactly those bytes to the connection decoder
== 0	Peer performed orderly shutdown	Treat as disconnect
< 0 + EINTR	Interrupted by signal	Retry
< 0 + EAGAIN/EWOULDBLOCK	No bytes available now	Return to poll()
< 0 + other errno	Real error	Handle/close connection

TCP is a byte stream: one send() is not one recv(). A frame may be split across calls, and multiple frames may arrive together. GCMD/GCOE headers and stream decoders restore message boundaries.

9. Socket options for Slipstream

Option	Where	Recommendation
SO_REUSEADDR	Listening sockets, before bind()	Use
TCP_NODELAY	Connected TCP sockets	Use for small latency-sensitive frames
SO_KEEPALIVE	Connected TCP sockets	Optional defensive liveness
SO_RCVBUF	Kernel receive queue	Leave default until measured
SO_SNDBUF	Kernel send queue	Leave default until measured
SO_REUSEPORT	Multiple listeners sharing endpoint	Not needed
SO_LINGER	Close behavior	Avoid unless a concrete protocol need exists

TCP_NODELAY

```
int enabled = 1;
::setsockopt(fd, IPPROTO_TCP, TCP_NODELAY,
             &enabled, sizeof(enabled));
```

It disables the Nagle algorithm and reduces delays for small frames. It changes latency behavior, not correctness.

SO_KEEPALIVE and application Heartbeat

```
TCP keepalive       asks whether the TCP peer/network is reachable
GCMD Heartbeat      asks whether the application protocol is alive
```

Keepalive defaults can be very long. Slipstream should still process Heartbeat messages, recv() == 0, and poll hangup/error events.

Buffers

```
Application bytes -> send() -> kernel SO_SNDBUF -> network
Network -> kernel SO_RCVBUF -> recv() -> application buffer -> decoder
```

Kernel buffers do not replace userspace buffers and do not eliminate partial I/O. Linux already tunes TCP buffering; measure before changing it.

10. shutdown() versus close()

shutdown(): change communication state

```
::shutdown(fd, SHUT_RD);    // stop receiving
::shutdown(fd, SHUT_WR);    // stop sending; peer eventually sees EOF
::shutdown(fd, SHUT_RDWR);  // stop both directions
```

SHUT_WR creates a half-close: this endpoint is finished sending but may continue receiving. shutdown() operates on the full-duplex connection state.

close(): release descriptor ownership

```
Socket::~Socket() {
    if (fd_ != -1) {
        ::close(fd_);
    }
}
```

After close(), the numeric descriptor may be reused for an unrelated resource. A move-only RAII Socket should set moved-from or explicitly closed descriptors to -1 and must never throw from its destructor.

Operation	Purpose
shutdown()	Explicitly disable reading/writing on the connection
close()	Release this process's descriptor reference

11. RAII Socket design checklist

```
class Socket {
public:
    Socket();
    ~Socket();

    Socket(const Socket&) = delete;
    Socket& operator=(const Socket&) = delete;

    Socket(Socket&&) noexcept;
    Socket& operator=(Socket&&) noexcept;

    void SetSocketOption(int level, int option, int value);
    void Bind(std::uint16_t port);
    void Listen(int backlog);
    [[nodiscard]] Socket Accept();
    void Connect(std::string_view ip, std::uint16_t port);

    [[nodiscard]] int NativeHandle() const noexcept;

private:
    explicit Socket(int adopted_fd, AdoptFdTag);
    int fd_{-1};
};
```

- The normal constructor calls socket().
- The private adopt constructor wraps an fd already returned by accept().
- Copying is deleted to prevent two destructors from closing the same fd.
- Moving transfers fd ownership and resets the source to -1.
- The destructor closes a valid fd and never throws.
- Application code remains responsible for calling Bind, Listen, Accept, or Connect in a valid order.

12. Recommended M2 architecture

Listeners

```
socket(SOCK_CLOEXEC | SOCK_NONBLOCK)
setsockopt(SO_REUSEADDR)
bind()
listen()
poll(POLLIN)
accept4(SOCK_CLOEXEC | SOCK_NONBLOCK)
```

Connected sockets

```
setsockopt(TCP_NODELAY)
optionally setsockopt(SO_KEEPALIVE)
poll(POLLIN and only-needed POLLOUT)
recv() until EAGAIN
send() remaining output until complete or EAGAIN
```

Per-connection state

```
MD connection: MarketDataStreamDecoder + pending output/state
OE connection: Order/market stream decoder + pending GCOE output
```

When either client disconnects because of session close or CSV exhaustion, Slipstream closes the other connected socket and prints completion details. If exactly one client of each type is allowed for the run, the listening sockets may be closed after both accepts.

13. Final API cheat sheet

API	Purpose
socket()	Create a kernel socket and return an fd
setsockopt()	Configure socket or protocol behavior
inet_pton()	Convert numeric IP text into binary representation
htons()	Convert a 16-bit port to network byte order
bind()	Assign a local address and port
listen()	Turn a TCP socket into a listener
accept()/accept4()	Return a new connected server-side socket
connect()	Establish a client-side peer connection
fcntl(O_NONBLOCK)	Configure persistent non-blocking behavior
poll()	Wait for readiness on multiple descriptors
getsockopt(SO_ERROR)	Read the result of non-blocking connect
send()	Transmit connected-socket bytes with optional flags
recv()	Receive currently available connected-socket bytes
shutdown()	Disable part or all of a full-duplex connection
close()	Release descriptor ownership

Documentation

Linux manual pages: man 2 socket; man 2 setsockopt; man 7 socket; man 7 tcp; man 7 ip; man 2 bind; man 2 listen; man 2 accept; man 2 connect; man 2 poll; man 2 send; man 2 recv; man 2 shutdown; man 2 close.

Online reference: https://man7.org/linux/man-pages/

recv() return-value quick reference

RESULT	MEANING	ACTION
`> 0`	That many bytes arrived	Feed exactly those bytes to the stream decoder
`== 0`	Peer closed its sending side	Treat as orderly disconnect / EOF
`< 0, EINTR`	A signal interrupted recv()	Retry recv()
`< 0, EAGAIN/EWOULDBLOCK`	No bytes are available right now	Return to poll(); connection is still alive
`< 0, other errno`	A real socket error occurred	Handle or close the connection

Key distinction: EAGAIN means try later; recv() == 0 means the peer disconnected.